

ЛАБОРАТОРНА РОБОТА № 3

Хід роботи:

3.1. Завдання 1: Ініціалізація проєкту

Створіть новий Node.js-проєкт з усіма необхідними залежностями.

Вимоги:

- Ініціалізуйте проєкт (npm init -y) та встановіть залежності:
runtime: express, cors, zod
dev: typescript, @types/* для кожного пакету, ts-node, jest, ts-jest, @types/jest, supertest, @types/supertest
- Налаштуйте tsconfig.json та jest.config.js.
- Додайте до package.json скрипти: dev, build, test, test: coverage.
- Структура проєкту:
project/
src/
schemas/entity.schema.ts
storage/entity.ts
routes/entity.ts
middleware/
app.ts
server.ts
tests/entity.test.ts
tsconfig.json
jest.config.js
package.json

					ДУ «Житомирська політехніка».25.121.13.000 – Лр1						
Змн.	Арк.	№ докум.	Підпис	Дата							
Розроб.		Крамар Д. О.			Звіт з лабораторної роботи			Лім.	Арк.	Аркушів	
Перевір.		Фуріхата Д. В.								1	ZZ
Керівник								ФІКТ Гр. ІПЗ-24-3[1]			
Н. контр.											
Зав. каф.											

```
// package.json
{
  "name": "nodejs3",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "dev": "ts-node src/server.ts",
    "build": "tsc",
    "test": "jest",
    "test:coverage": "jest --coverage"
  },
  "dependencies": {
    "cors": "^2.8.6",
    "dotenv": "^17.4.1",
    "express": "^5.2.1",
    "zod": "^4.3.6"
  },
  "devDependencies": {
    "@types/cors": "^2.8.19",
    "@types/express": "^5.0.6",
    "@types/jest": "^30.0.0",
    "@types/node": "^25.5.0",
    "@types/supertest": "^7.2.0",
    "jest": "^30.3.0",
    "supertest": "^7.2.2",
    "ts-jest": "^29.4.6",
    "ts-node": "^10.9.2",
    "typescript": "^5.9.3"
  }
}
```

```
// tsconfig.json
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "CommonJS",
    "rootDir": "./src",
    "outDir": "./dist",
  }
}
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

```

"strict": true,
"esModuleInterop": true,
"skipLibCheck": true,
"forceConsistentCasingInFileNames": true,
"types": ["node", "jest"]
},
"include": ["src/**/*.ts"],
"exclude": ["node_modules", "dist", "**/*.test.ts"]
}

// jest.config.js
/** @type {import('ts-jest').JestConfigWithTsJest} */
module.exports = {
  preset: 'ts-jest',
  testEnvironment: 'node',
  roots: ['<rootDir>/src'],
  testMatch: ['**/*.test.ts', '**/*.spec.ts'],
  collectCoverageFrom: ['src/**/*.ts', '!src/**/*.d.ts'],
  coverageDirectory: 'coverage',
};

```

3.2. Завдання 2: Схеми та типи сутності

Створіть схему для роботи з обраною сутністю. Оберіть сутність із домену, який вам знайомий — книга, рецепт, фільм, гравець, планета тощо. Чим краще ви розумієте предметну область, тим легше вийде зробити схему. Не треба повторювати приклад з лекції :)

Вимоги:

- Схема створення (createSchema) повинна містити:
 - обов'язкове текстове поле з назвою, що відповідає вашій сутності (title для книги, пате для персонажа, designation для планети тощо) 1-100 символів;
 - опціональне поле опису до 500 символів;
 - мінімум 2 додаткових поля, що відображають реальні атрибути вашої сутності. Поле має відповідати на питання: «що корисно знати про цей об'єкт?» Наприклад, для книги це може бути рік видання (число зі зрозумілими обмеженнями) та жанр (список заготовлених значень); для рецепту — час приготування в хвилинах та рівень складності. Поля типу isActive чи count без конкретного змісту не зараховуються.
- Схема оновлення (updateSchema) — всі поля опціональні.
- Тип збереженої сутності (Entity) — виводиться з createSchema (z.infer) і

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Пр1	Арк.
		Фуріхата Д. В.				3
Змн.	Арк.	№ докум.	Підпис	Дата		

розширюється серверними полями: id, createdAt, updatedAt.

```
// category.schema.ts
import { z } from 'zod';

export const createCategorySchema = z.object({
  name: z.string().min(1, "Назва не може бути порожньою").max(100, "Назва за-
надто довга"),
  description: z.string().max(500, "Опис занадто довгий").optional(),
  type: z.enum(['income', 'expense']),
  color: z.string().regex(/^#[0-9A-Fa-f]{6}$/, "Невірний формат кольору, очіку-
ється HEX (наприклад, #FF0000)",
  userId: z.number().int().positive("ID користувача має бути додатним числом")
});

export const updateCategorySchema = createCategorySchema.partial();

export type CreateCategoryInput = z.infer<typeof createCategorySchema>;
export type UpdateCategoryInput = z.infer<typeof updateCategorySchema>;

export type CategoryEntity = CreateCategoryInput & {
  id: string;
  createdAt: Date;
  updatedAt: Date;
};
```

3.3. Завдання 3: Модуль зберігання (in-memory storage)

Реалізуйте CRUD-модуль на основі Map.

Вимоги:

- Реалізуйте модуль на основі Map<string, Entity>. Назви функцій та їх сигна-тури — на ваш розсуд.
- Модуль повинен підтримувати: отримання всіх записів, отримання за id, створення, оновлення (з оновленням updatedAt), видалення.
- Передбачте можливість скидання стану сховища — вона знадобиться для ізо-ляції тестів.

```
// InMemoryCategoryRepository.ts
import { ICategoryRepository, PaginatedResult, PaginationOptions } from
'./ICategoryRepository';
import { CategoryEntity, CreateCategoryInput, UpdateCategoryInput } from
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Пр1	Арк.
		Фуріхата Д. В.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

```

'../../schemas/category.schema';
import crypto from 'crypto';

export class InMemoryCategoryRepository implements ICategoryRepository {
  private storage = new Map<string, CategoryEntity>();

  async findMany(filters?: { userId?: number; type?: string }, options?:
PaginationOptions): Promise<PaginatedResult<CategoryEntity>> {
    let results = Array.from(this.storage.values());

    if (filters) {
      if (filters.userId !== undefined) {
        results = results.filter(category => category.userId === filters.userId);
      }
      if (filters.type !== undefined) {
        results = results.filter(category => category.type === filters.type);
      }
    }

    const page = options?.page || 1;
    const limit = options?.limit || 10;
    const total = results.length;
    const data = results.slice((page - 1) * limit, page * limit);

    return {
      data,
      pagination: { page, limit, total, totalPages: Math.ceil(total / limit) }
    };
  }

  async findById(id: string): Promise<CategoryEntity | null> {
    return this.storage.get(id) || null;
  }

  async create(data: CreateCategoryInput): Promise<CategoryEntity> {
    const id = crypto.randomUUID();
    const now = new Date();
    const newEntity: CategoryEntity = { ...data, id, createdAt: now, updatedAt: now
  };

```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    this.storage.set(id, newEntity);
    return newEntity;
  }

  async updateById(id: string, data: UpdateCategoryInput): Promise<CategoryEntity
  | null> {
    const existing = this.storage.get(id);
    if (!existing) return null;
    const updated = { ...existing, ...data, updatedAt: new Date() };
    this.storage.set(id, updated as CategoryEntity);
    return updated as CategoryEntity;
  }

  async deleteById(id: string): Promise<boolean> {
    return this.storage.delete(id);
  }

  reset(): void {
    this.storage.clear();
  }
}

```

3.4. Завдання 4: CRUD-маршрути та middleware

Реалізуйте Express Router з повним набором CRUD-маршрутів, validate middleware та errorHandler.

Вимоги:

- Реалізуйте validate (schema) middleware — парсить тіло запиту через схему і викидає помилку, якщо дані невалідні.
- Реалізуйте errorHandler — розрізняйте помилки валідації (ZodError, статус 400) та інші помилки (статус 500). Формат відповіді при помилці — на ваш розсуд, але він має бути інформативним.
- Підтримайте операції: отримання всіх записів, отримання за id, створення, оновлення, видалення.
- Обирайте статус-коди відповідно до HTTP-семантики: 200, 201, 204, 400, 404 тощо.
- При створенні генеруйте id через crypto.randomUUID(), виставляйте дату createdAt/updatedAt.

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				
Змн.	Арк.	№ докум.	Підпис	Дата		6

```
// validate.ts
import { Request, Response, NextFunction } from 'express';
import { ZodObject } from 'zod';

export const validate = (schema: ZodObject<any>) =>
  async (req: Request, res: Response, next: NextFunction) => {
    try {
      req.body = await schema.parseAsync(req.body);
      next();
    } catch (error) {
      next(error);
    }
  };

// errorHandler.ts
import { Request, Response, NextFunction } from 'express';
import { ZodError } from 'zod';

export const errorHandler = (err: Error, req: Request, res: Response, next:
NextFunction) => {
  if (err instanceof ZodError) {
    res.status(400).json({
      status: 'error',
      message: 'Помилка валідації даних запиту',
      issues: err.issues.map(e => ({
        field: e.path.join('.'),
        message: e.message
      })))
  });
  return;
}

console.error('[Unhandled Error]:', err);
res.status(500).json({
  status: 'error',
  message: 'Внутрішня помилка сервера'
});
};
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

```
// category.routes.ts
import { Router, Request, Response, NextFunction } from 'express';
import { validate } from '../middleware/validate';
import { createCategorySchema, updateCategorySchema } from
'../schemas/category.schema';
import { CategoryService } from '../storage/services/CategoryService';

export const createCategoryRouter = (categoryService: CategoryService) => {
  const router = Router();

  router.get('/', async (req: Request, res: Response, next: NextFunction) => {
    try {
      const { userId, type, page, limit, sort } = req.query;
      const filters = { userId: userId ? Number(userId) : undefined, type: type as
string | undefined };
      const options = { page: page ? Number(page) : 1, limit: limit ? Number(limit)
: 10, sort: sort as string | undefined };
      const result = await categoryService.getAll(filters, options);
      res.status(200).json(result);
    } catch (error) { next(error); }
  });

  router.get('/:id', async (req: Request<{ id: string }>, res: Response, next:
NextFunction) => {
    try {
      const category = await categoryService.getById(req.params.id);
      if (!category) {
        res.status(404).json( { error: 'Категорію не знайдено' } );
        return;
      }
      res.status(200).json(category);
    } catch (error) { next(error); }
  });

  router.post('/', validate(createCategorySchema), async (req: Request, res:
Response, next: NextFunction) => {
    try {
      const newCategory = await categoryService.create(req.body);
      res.status(201).json(newCategory);
    }
  });
}
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				8
Змн.	Арк.	№ докум.	Підпис	Дата		


```
    } catch (error) { next(error); }
  });
```

```
  router.patch('/:id', validate(updateCategorySchema), async (req: Request<{ id:
string }>, res: Response, next: NextFunction) => {
    try {
      const updatedCategory = await categoryService.update(req.params.id,
req.body);
      if (!updatedCategory) {
        res.status(404).json({ error: 'Категорію не знайдено' });
        return;
      }
      res.status(200).json(updatedCategory);
    } catch (error) { next(error); }
  });
```

```
  router.delete('/:id', async (req: Request<{ id: string }>, res: Response, next:
NextFunction) => {
    try {
      const isDeleted = await categoryService.delete(req.params.id);
      if (!isDeleted) {
        res.status(404).json({ error: 'Категорію не знайдено' });
        return;
      }
      res.status(204).send();
    } catch (error) { next(error); }
  });
```

```
  return router;
};
```

3.5. Завдання 5: Розділення app.ts та server.ts

Розділіть конфігурацію Express та запуск сервера.

Вимоги:

- app.ts - створює і налаштовує Express, не викликає listen(), експортує app.
- server.ts — імпортує app, викликає listen().
- Дотримуйтесь правильного порядку підключення middleware.

```
// app.ts
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				9
Змн.	Арк.	№ докум.	Підпис	Дата		

```
import express, { Request, Response } from 'express';
import cors from 'cors';
import { createCategoryRouter } from './routes/category.routes';
import { errorHandler } from './middleware/errorHandler';
import { InMemoryCategoryRepository } from
'./storage/repositories/InMemoryCategoryRepository';
import { CategoryService } from './storage/services/CategoryService';
```

```
const app = express();
```

```
app.get('/health', (req: Request, res: Response) => {
  res.status(200).json({ status: 'ok' });
});
```

```
app.use(cors());
app.use(express.json());
```

```
export const repository = new InMemoryCategoryRepository();
export const service = new CategoryService(repository);
export const categoryRouter = createCategoryRouter(service);
```

```
app.use('/categories', categoryRouter);
app.use(errorHandler);
```

```
export default app;
```

```
// server.ts
```

```
import app from './app';
import dotenv from 'dotenv';
```

```
dotenv.config();
```

```
const PORT = process.env.PORT || 3000;
```

```
async function startServer() {
  try {
    const server = app.listen(PORT, () => {
      console.log(`http://localhost:${PORT}`);
    });
  } catch (error) {
    console.error(error);
  }
}
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				10
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    });

    const gracefulShutdown = async (signal: string) => {
      console.log(`\n...`);
      server.close() => {
        console.log('Stopped.');
```

```

        process.exit(0);
      });
    };

    process.on('SIGTERM', () => gracefulShutdown('SIGTERM'));
    process.on('SIGINT', () => gracefulShutdown('SIGINT'));

  } catch (error) {
    console.error('Error starting server:', error);
    process.exit(1);
  }
}

startServer();
```

3.6. Завдання 6: Інтеграційні тести з Supertest

Напишіть інтеграційні тести для вашого API.

Вимоги:

- Використовуйте `beforeEach` для очистки пам'яті та ізоляції тестів.
- Покрийте тестами всі маршрути: успішні сценарії та граничні випадки (не знайдено, невалідні дані тощо).
- Перевіряйте статус-коди, структуру відповідей і заголовки там, де це доречно. Загальна кількість тестів не менше 12.

```

// category.test.ts
import request from 'supertest';
import app, { repository } from '../app';

beforeEach(() => {
  repository.reset();
});

describe('Category API Integration Tests', () => {
```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				11
Змн.	Арк.	№ докум.	Підпис	Дата		

```

const validCategory = {
  name: 'Продукти',
  description: 'Витрати на супермаркет',
  type: 'expense',
  color: '#FF0000',
  userId: 1
};

it('повинен створити категорію та повернути статус 201', async () => {
  const res = await request(app)
    .post('/categories')
    .send(validCategory)
    .expect(201);

  expect(res.body).toHaveProperty('id');
  expect(res.body.name).toBe(validCategory.name);
});

it('повинен повернути 400 при невалідних даних (Zod)', async () => {
  const invalidData = { name: '', type: 'wrong_type' };
  const res = await request(app)
    .post('/categories')
    .send(invalidData)
    .expect(400);

  expect(res.body.status).toBe('error');
});

it('повинен повернути список категорій з пагінацією зі статусом 200', async ()
=> {
  await request(app).post('/categories').send(validCategory);

  const res = await request(app)
    .get('/categories')
    .expect(200);

  expect(Array.isArray(res.body.data)).toBe(true);
  expect(res.body.data.length).toBe(1);
});

```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				12
Змн.	Арк.	№ докум.	Підпис	Дата		

```

it('повинен повернути запис за ID зі статусом 200', async () => {
  const createRes = await request(app).post('/categories').send(validCategory);
  const categoryId = createRes.body.id;

  const getRes = await request(app)
    .get(`/categories/${categoryId}`)
    .expect(200);

  expect(getRes.body.id).toBe(categoryId);
});

it('повинен частково оновити запис зі статусом 200', async () => {
  const createRes = await request(app).post('/categories').send(validCategory);
  const categoryId = createRes.body.id;

  const updateRes = await request(app)
    .patch(`/categories/${categoryId}`)
    .send({ name: 'Нова назва', color: '#00FF00' })
    .expect(200);

  expect(updateRes.body.name).toBe('Нова назва');
  expect(updateRes.body.color).toBe('#00FF00');
});

it('повинен видалити запис та повернути 204 No Content', async () => {
  const createRes = await request(app).post('/categories').send(validCategory);
  const categoryId = createRes.body.id;

  await request(app).delete(`/categories/${categoryId}`).expect(204);
  await request(app).get(`/categories/${categoryId}`).expect(404);
});

```

```

Test Suites: 1 passed, 1 total
Tests:      19 passed, 19 total
Snapshots:  0 total
Time:       5.001 s

```

Рис. 1.

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				13
Змн.	Арк.	№ докум.	Підпис	Дата		

3.7. Завдання 7: Фільтрація через query-параметри

Розширте додаток підтримкою фільтрації та напишіть тести.

Фільтрацію слід реалізувати на рівні сховища, а не в обробнику маршруту: функція отримання всіх записів приймає опціональні параметри та повертає вже відфільтрований результат. Це правильний поділ відповідальності — у наступних темах, коли сховище замінимо на базу даних, фільтрація природно перетвориться на умови запиту, не змінюючи код маршрутів.

Вимоги:

- Реалізуйте щонайменше 2 query-параметри для фільтрації/пошуку, що мають сенс для вашої предметної області. Назви параметрів обирайте самостійно.
- Параметри можна комбінувати.
- Напишіть тести для кожного параметра та їх комбінацій.
- Додайте один додатковий маршрут з власним шляхом, специфічний для вашої предметної області. Наприклад: GET /planets/habitable, GET /books/bestsellers, GET /recipes/quick. Покрийте його тестами.

```
// CategoryService.ts
```

```
import { ICategoryRepository, PaginatedResult, PaginationOptions } from
'../repositories/ICategoryRepository';
import { CategoryEntity, CreateCategoryInput, UpdateCategoryInput } from
'../schemas/category.schema';
```

```
export class CategoryService {
  constructor(private repository: ICategoryRepository) {}
```

```
  async getAll(
    filters?: { userId?: number; type?: string },
    options?: PaginationOptions
  ): Promise<PaginatedResult<CategoryEntity>> {
    return this.repository.findMany(filters, options);
  }
}
```

```
// Додатковий маршрут у category.routes.ts
```

```
router.get('/expenses', async (req: Request, res: Response, next: NextFunction) =>
{
  try {
    const { page, limit, sort } = req.query;
    const options = { page: page ? Number(page) : 1, limit: limit ? Number(limit)
: 10, sort: sort as string | undefined };

```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				14
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    const result = await categoryService.getAll({ type: 'expense' }, options);
    res.status(200).json(result);
  } catch (error) {
    next(error);
  }
});

// комбінації фільтрів category.test.ts
it('повинен комбінувати фільтри за type та userId', async () => {
  await request(app).post('/categories').send({ ...validCategory, type: 'expense',
userId: 1 });
  await request(app).post('/categories').send({ ...validCategory, type: 'income',
userId: 1 });
  await request(app).post('/categories').send({ ...validCategory, type: 'expense',
userId: 2 });

  const res = await
request(app).get('/categories?type=expense&userId=1').expect(200);

  expect(res.body.data.length).toBe(1);
  expect(res.body.data[0].type).toBe('expense');
  expect(res.body.data[0].userId).toBe(1);
});

it('повинен повертати дані зі специфічного маршруту /expenses', async () => {
  await request(app).post('/categories').send({ ...validCategory, type: 'expense' });
  await request(app).post('/categories').send({ ...validCategory, type: 'income' });

  const res = await request(app).get('/categories/expenses').expect(200);

  expect(Array.isArray(res.body.data)).toBe(true);
  expect(res.body.data[0].type).toBe('expense');
});

```

		Крамар Д. О.			ДУ «Житомирська політехніка».25.121.13.000 – Лр1	Арк.
		Фуріхата Д. В.				15
Змн.	Арк.	№ докум.	Підпис	Дата		